

GRADUAÇÃO

 FIAP

# MOBILE APPLICATION DEVELOPMENT

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS



## AULA 06

---

Quem está usando o aplicativo?

Autenticação, sessão e JWT



“*Quem sou eu neste mundo? Ah, esse é o grande enigma.*”  
— *Lewis Carroll, Alice no País das Maravilhas*



# O APLICATIVO SABE BUSCAR E ALTERAR DADOS



**MAS AINDA FALTA UMA INFORMAÇÃO IMPORTANTE.**

# DE QUEM SÃO ESSES DADOS?

Ana  → GET /tweets → API  
Bruno 

Sem alguma forma de identificação, a API não consegue distinguir as duas pessoas.

# UM IDENTIFICADOR NÃO COMPROVA UMA IDENTIDADE

Ana —————→ GET /tweets?userId=1 —→ API  
Bruno —————→ GET /tweets?userId=2 —→ API

# A API PRECISA CONFIRMAR QUEM ESTÁ POR TRÁS DO IDENTIFICADOR

Precisamos coletar as credenciais e as envia-lás para verificação.

```
usuário + senha
↓
POST /auth/login
↓
API
↓
aceita ou recusa
```

**ESSE PROCESSO É CHAMADO DE AUTENTICAÇÃO.**

# REPETIR AS CREDENCIAIS EM CADA REQUISIÇÃO NÃO É A SOLUÇÃO

Depois que a identidade foi confirmada, o aplicativo ainda precisa:

1. guardar o resultado;
2. usá-lo nas próximas requisições.

**A SENHA NÃO DEVE SER REENVIADA PARA BUSCAR DADOS PERTENCENTES AO USUÁRIO.**

# COMO O RESULTADO DO LOGIN ACOMPANHA AS PRÓXIMAS REQUISIÇÕES?

# OBJETIVOS

Ao final da aula, você será capaz de:

- diferenciar autenticação, sessão e JWT;
- explicar por que o conteúdo de um JWT não é secreto;
- executar o login com `useMutation`;
- manter uma sessão durante o uso do aplicativo;
- enviar o token em uma consulta protegida.

# TRÊS CONCEITOS DISTINTOS

Autenticação, sessão e JWT têm responsabilidades diferentes.

| Conceito     | Responsabilidade                                  |
|--------------|---------------------------------------------------|
| Autenticação | Confirmar quem é o usuário                        |
| Sessão       | Manter o estado autenticado entre as requisições  |
| JWT          | Representar uma credencial em um formato compacto |

# CONFIRMAÇÃO DE IDENTIDADE

A autenticação confirma a identidade do usuário.

## AUTENTICAÇÃO

usuário + senha  
↓  
servidor confere  
↓  
aceita ou recusa

**As credenciais são usadas para confirmar a identidade no login.**

# CONTINUIDADE DA SESSÃO

**A PARTIR DA IDENTIDADE CONFIRMADA, A SESSÃO MANTÉM O ESTADO AUTENTICADO**

```
SESSÃO
usuário autenticado
      +
credencial recebida
      ↓
próximas requisições
```

# JWT É UM FORMATO DE TOKEN

JWT significa **JSON Web Token**.

Ele pode transportar informações — chamadas de *claims* — em um formato compacto.

autenticação ≠ sessão ≠ JWT

**NEM TODA AUTENTICAÇÃO USA JWT E NEM TODO TOKEN PRECISA SER UM JWT**

# DO FORMULÁRIO ATÉ OS DADOS PROTEGIDOS

```
usuário + senha
  ↓
POST /auth/login
  ↓
accessToken
  ↓
sessão
  ↓
Authorization: Bearer token
  ↓
API
```

# O LOGIN COMEÇA COM UMA REQUISIÇÃO

```
const response = await fetch("https://dummyjson.com/auth/login", {  
  method: "POST",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify({  
    username: "emilys",  
    password: "emilypass",  
  }),  
});
```

# A RESPOSTA CONTÉM UMA CREDENCIAL

```
{  
  "id": 1,  
  "firstName": "Emily",  
  "email": "emily.johnson@x.dummyjson.com",  
  "accessToken": "eyJhbGciOiJIUzI1NiIs...",  
  "refreshToken": "eyJhbGciOiJIUzI1NiIs..."  
}
```

Hoje usaremos apenas o `accessToken`.

# O JWT POSSUI TRÊS PARTES

header.payload.signature

## Header

algoritmo e tipo

## Payload

claims

## Signature

proteção de integridade

# CODIFICADO NÃO SIGNIFICA CRIPTOGRAFADO

O `header` e o `payload` usam Base64URL.

Isso significa que qualquer pessoa que tenha o token pode decodificá-los.

Senhas, chaves privadas e outros segredos não devem ser colocados no payload.

Ler o token não prova que ele é verdadeiro. A API ainda precisa verificar sua assinatura e validade.

# O TOKEN FUNCIONA COMO UMA CREDENCIAL AO PORTADOR

**Authorization:** Bearer eyJhbGciOiJIUzI1NiIs...

Quem possui um token ao portador pode apresentá-lo.

Por isso:

- o token precisa ser protegido;
- a comunicação precisa usar HTTPS;
- o token não deve aparecer em logs.

# CONSULTAS PROTEGIDAS USAM A CREDENCIAL DA SESSÃO

```
const response = await fetch("https://dummyjson.com/auth/me", {  
  headers: {  
    Authorization: `Bearer ${accessToken}`,  
  },  
});
```

Sem token ou com um token inválido, a API recusa a requisição.

# AUTENTICAR E AUTORIZAR SÃO DECISÕES DIFERENTES

Autenticação  
“Quem é você?”

Autorização  
“O que você pode fazer?”

**HOJE NOSSO FOCO ESTÁ NA PRIMEIRA PERGUNTA**

# ONDE O TANSTACK QUERY ENTRA NESSE FLUXO?

| Operação        | Recurso                  | Motivo                                      |
|-----------------|--------------------------|---------------------------------------------|
| Entrar          | <code>useMutation</code> | Acontece após o envio do formulário         |
| Buscar o perfil | <code>useQuery</code>    | Representa um dado remoto consultável       |
| Sair            | Sessão                   | Remove a credencial mantida pelo aplicativo |

# LOGIN É UMA OPERAÇÃO DISPARADA PELO USUÁRIO

```
export const useLoginMutation = () => {  
  return useMutation({  
    mutationFn: login,  
  });  
};
```

Mesmo sem alterar um recurso de negócio, o login é uma operação imperativa: recebe credenciais e só executa após uma ação do usuário.

# O APLICATIVO INICIA A SESSÃO APÓS O SUCESSO

```
loginMutation.mutate(  
  { username, password },  
  {  
    onSuccess: ({ id, accessToken }) => {  
      criarSessao({  
        userId: id,  
        accessToken,  
      });  
      router.redirect("/");  
    },  
  },  
);
```

A senha não é guardada na sessão.

# A SESSÃO REPRESENTA O USUÁRIO AUTENTICADO

```
type Sessao = {  
  userId: number;  
  accessToken: string;  
};  
  
const [sessao, setSessao] = useState<Sessao | null>(null);  
  
const criarSessao = (novaSessao: Sessao) => setSession(novaSessao);  
  
const sair = () => setSessao(null);
```

# A CONSULTA DO PERFIL DEPENDE DA SESSÃO

```
export function usePerfilQuery() {  
  const { sessao } = useSessao();  
  
  return useQuery({  
    queryKey: ["perfil", sessao?.userId],  
    queryFn: () => getUserProfile(sessao?.token ?? ""),  
    enabled: sessao !== null,  
  });  
}
```

• O token não precisa fazer parte da `queryKey`.

# A SESSÃO COORDENA A INTERFACE

```
const { sessao } = useSessao();

if (!sessao) {
  return <Redirect href="/login" />;
} else {
  return <Redirect href="/perfil" />;
}
```

Enquanto isso, mutation e query continuam representando seus próprios estados de carregamento e erro.



## APLICATIVO “AUTH LAB”

Um fluxo completo de autenticação com TanStack Query

[Acesse o Codelab](#)

## O QUE VOCÊ VAI APRENDER?

- formulário de login;
- mutation para autenticar;
- sessão compartilhada pelo aplicativo;
- consulta autenticada do perfil;
- ação para sair.

# SÍNTESE

- login é um evento
- sessão é um estado
- JWT é uma credencial
- a API valida a identidade



## O LIVRO DOS CÓDIGOS

**Simon Singh**

+ Uma história acessível sobre como diferentes sociedades transformaram segredo, confiança e  
| identidade em mecanismos verificáveis.

# REFERÊNCIAS

- RFC 7519 — JSON Web Token
- RFC 6750 — Bearer Token Usage
- DummyJSON — Authentication
- TanStack Query — `useMutation`
- TanStack Query — `useQuery`
- Expo — SecureStore
- Expo Router — Authentication

# LICENÇA

Este material é disponibilizado sob a licença Creative Commons Atribuição 4.0 Internacional (CC BY 4.0).

  Atribuição 4.0 Internacional

Autor: Felipe Cavalcante Vilella de Oliveira, 2026